Kế hoạch triển khai Walkie-Talkie Offline App

> Mini Project — Java + XML — Android Wi-Fi Direct

🗺️ Tổng quan kiến trúc

Trước khi vào từng phase, cần hiểu rõ luồng hoạt động:

```
[Máy A] ---(Wi-Fi Direct)--- [Máy B]
   |                            |
Group Owner (Server)          Client
ServerSocket(8888)          Socket → connect(GO_IP, 8888)
   |                            |
SendAudioThread            SendAudioThread
ReceiveAudioThread         ReceiveAudioThread
```

Nguyên tắc cốt lõi: Wi-Fi Direct tạo ra một mạng nội bộ tạm thời giữa các thiết bị mà **không cần router/Internet**. Một máy sẽ đóng vai Group Owner (như router mini), các máy còn lại là Client.

✅ Phase 1 — Cấu trúc Project & UI

1.1 Cấu trúc thư mục đề xuất

```
app/
├── manifests/
│   └── AndroidManifest.xml
├── java/com.example.walkietalkie/
│   ├── MainActivity.java
│   ├── wifi/
│   │   ├── WiFiDirectBroadcastReceiver.java
│   │   ├── ServerClass.java
│   │   └── ClientClass.java
│   ├── audio/
│   │   ├── SendAudioThread.java
│   │   └── ReceiveAudioThread.java
│   └── adapter/
│       └── PeerListAdapter.java
```

```
└── res/
    └── layout/
        └── activity_main.xml
```

1.2 AndroidManifest.xml — Khai báo quyền

```xml
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.NEARBY_WIFI_DEVICES"
    android:usesPermissionFlags="neverForLocation"
    tools:targetSdkVersion="33" />
<uses-permission android:name="android.permission.RECORD_AUDIO" />
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_WIFI_STATE" />
<uses-permission android:name="android.permission.CHANGE_WIFI_STATE" />
<uses-permission android:name="android.permission.CHANGE_NETWORK_STATE" />
```

> ⚠️ Thiếu `CHANGE_WIFI_STATE` và `ACCESS_WIFI_STATE` là app sẽ crash ngay — tài liệu gốc bỏ sót 2 cái này.

1.3 Layout activity_main.xml (ConstraintLayout)

Gồm 4 thành phần chính:

- `tvStatus` — hiển thị trạng thái (Disconnected / Discovering / Connected)
- `btnDiscover` — bắt đầu quét thiết bị
- `rvPeers` — danh sách thiết bị tìm được (RecyclerView)
- `btnTalk` — nút bấm giữ để nói (Push-to-Talk)

1.4 Logic xin quyền động trong MainActivity

```java
private void requestPermissions() {
    List<String> permissions = new ArrayList<>();

    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
        // Android 13+ (API 33)
        permissions.add(Manifest.permission.NEARBY_WIFI_DEVICES);
    } else {
        // Dưới Android 13
        permissions.add(Manifest.permission.ACCESS_FINE_LOCATION);
    }
    permissions.add(Manifest.permission.RECORD_AUDIO);

    // Lọc ra những quyền chưa được cấp
    List<String> toRequest = permissions.stream()
```

```
        .filter(p -> ContextCompat.checkSelfPermission(this, p)
                     != PackageManager.PERMISSION_GRANTED)
        .collect(Collectors.toList());

    if (!toRequest.isEmpty()) {
        ActivityCompat.requestPermissions(this,
            toRequest.toArray(new String[0]), REQUEST_CODE);
    }
}
```

✅ Phase 2 — Wi-Fi Direct Core (Khám phá & Lắng nghe)

2.1 Khởi tạo trong onCreate()

```
WifiP2pManager manager;
WifiP2pManager.Channel channel;
WiFiDirectBroadcastReceiver receiver;
IntentFilter intentFilter;

@Override
protected void onCreate(Bundle savedInstanceState) {
    manager = (WifiP2pManager) getSystemService(Context.WIFI_P2P_SERVICE);
    channel = manager.initialize(this, getMainLooper(), null);
    receiver = new WiFiDirectBroadcastReceiver(manager, channel, this);

    intentFilter = new IntentFilter();
    intentFilter.addAction(WifiP2pManager.WIFI_P2P_STATE_CHANGED_ACTION);
    intentFilter.addAction(WifiP2pManager.WIFI_P2P_PEERS_CHANGED_ACTION);

intentFilter.addAction(WifiP2pManager.WIFI_P2P_CONNECTION_CHANGED_ACTION);

intentFilter.addAction(WifiP2pManager.WIFI_P2P_THIS_DEVICE_CHANGED_ACTION);
}

@Override protected void onResume() { registerReceiver(receiver,
intentFilter); }
@Override protected void onPause()  { unregisterReceiver(receiver); }
```

2.2 WiFiDirectBroadcastReceiver — 4 sự kiện cần xử lý

Sự kiện	Việc cần làm
`WIFI_P2P_STATE_CHANGED_ACTION`	Kiểm tra Wi-Fi Direct có bật không, cập nhật UI
`WIFI_P2P_PEERS_CHANGED_ACTION`	Gọi `manager.requestPeers()` → cập nhật RecyclerView
`WIFI_P2P_CONNECTION_CHANGED_ACTION`	Kiểm tra kết nối → gọi `requestConnectionInfo()`
`WIFI_P2P_THIS_DEVICE_CHANGED_ACTION`	Cập nhật tên/địa chỉ máy hiện tại

2.3 PeerListAdapter cho RecyclerView

Adapter đơn giản hiển thị `device.deviceName` + `device.deviceAddress`.
Click vào item → trigger kết nối ở Phase 3.

✅ Phase 3 — Socket Connection (Client - Server)

3.1 Flow kết nối

```
User click vào peer trong RecyclerView
        ↓
manager.connect(channel, config, actionListener)
        ↓
BroadcastReceiver nhận WIFI_P2P_CONNECTION_CHANGED_ACTION
        ↓
manager.requestConnectionInfo(channel, this)
        ↓
onConnectionInfoAvailable(WifiP2pInfo info)
        ↓
info.isGroupOwner? ──YES──→ Chạy ServerClass (mở ServerSocket 8888)
                   ──NO───→ Chạy ClientClass (kết nối tới GO IP, port 8888)
```

3.2 ServerClass (chạy trên Thread riêng)

```java
public class ServerClass extends Thread {
    private ServerSocket serverSocket;
    public Socket socket;

    @Override
    public void run() {
        try {
```

```java
            serverSocket = new ServerSocket(8888);
            socket = serverSocket.accept(); // Chặn tại đây, chờ client kết nối
            // Sau khi accept() trả về → callback về MainActivity để bắt đầu audio
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

3.3 ClientClass (chạy trên Thread riêng)

```java
public class ClientClass extends Thread {
    public Socket socket;
    private String hostAddress;

    public ClientClass(InetAddress hostAddress) {
        this.hostAddress = hostAddress.getHostAddress();
        socket = new Socket();
    }

    @Override
    public void run() {
        try {
            socket.connect(new InetSocketAddress(hostAddress, 8888), 5000);
            // Kết nối thành công → callback về MainActivity để bắt đầu audio
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

> ⚠️ **Quan trọng:** Mọi thao tác Socket phải chạy trên **Thread riêng**, tuyệt đối không chạy trên Main Thread — sẽ crash `NetworkOnMainThreadException`.

✅ Phase 4 — Audio Streaming (Thu âm & Phát âm)

4.1 Thông số âm thanh — định nghĩa trong Constants.java

```java
public class Constants {
```

```
    public static final int SAMPLE_RATE  = 16000;
    public static final int CHANNEL_IN   = AudioFormat.CHANNEL_IN_MONO;
    public static final int CHANNEL_OUT  = AudioFormat.CHANNEL_OUT_MONO;
    public static final int AUDIO_FORMAT = AudioFormat.ENCODING_PCM_16BIT;
    public static final int BUFFER_SIZE  = AudioRecord.getMinBufferSize(
                                                SAMPLE_RATE, CHANNEL_IN,
AUDIO_FORMAT);
}
```

4.2 SendAudioThread — Thu âm và gửi đi

```
public class SendAudioThread extends Thread {
    private AudioRecord audioRecord;
    private OutputStream outputStream;
    private volatile boolean isRecording = false; // volatile để thread khác
đọc được

    public SendAudioThread(OutputStream os) {
        this.outputStream = os;
    }

    public void startRecording() { isRecording = true; }
    public void stopRecording()  { isRecording = false; }

    @Override
    public void run() {
        audioRecord = new AudioRecord(
            MediaRecorder.AudioSource.MIC,
            Constants.SAMPLE_RATE,
            Constants.CHANNEL_IN,
            Constants.AUDIO_FORMAT,
            Constants.BUFFER_SIZE
        );

        byte[] buffer = new byte[Constants.BUFFER_SIZE];
        audioRecord.startRecording();

        while (isRecording) {
            int bytesRead = audioRecord.read(buffer, 0, buffer.length);
            if (bytesRead > 0) {
                try {
                    outputStream.write(buffer, 0, bytesRead);
                    outputStream.flush();
                } catch (IOException e) {
                    break;
```

```
                }
            }
        }

        audioRecord.stop();
        audioRecord.release();
    }
}
```

4.3 ReceiveAudioThread — Nhận và phát ra loa

```
public class ReceiveAudioThread extends Thread {
    private AudioTrack audioTrack;
    private InputStream inputStream;

    public ReceiveAudioThread(InputStream is) {
        this.inputStream = is;
    }

    @Override
    public void run() {
        audioTrack = new AudioTrack.Builder()
            .setAudioAttributes(new AudioAttributes.Builder()
                .setUsage(AudioAttributes.USAGE_VOICE_COMMUNICATION)
                .setContentType(AudioAttributes.CONTENT_TYPE_SPEECH)
                .build())
            .setAudioFormat(new AudioFormat.Builder()
                .setSampleRate(Constants.SAMPLE_RATE)
                .setEncoding(Constants.AUDIO_FORMAT)
                .setChannelMask(Constants.CHANNEL_OUT)
                .build())
            .setBufferSizeInBytes(Constants.BUFFER_SIZE)
            .setTransferMode(AudioTrack.MODE_STREAM)
            .build();

        byte[] buffer = new byte[Constants.BUFFER_SIZE];
        audioTrack.play();

        while (!Thread.currentThread().isInterrupted()) {
            try {
                int bytesRead = inputStream.read(buffer);
                if (bytesRead > 0) audioTrack.write(buffer, 0, bytesRead);
            } catch (IOException e) {
                break;
            }
```

```
        }

        audioTrack.stop();
        audioTrack.release();
    }
}
```

4.4 Gắn Push-to-Talk vào btnTalk

```
btnTalk.setOnTouchListener((v, event) -> {
    if (event.getAction() == MotionEvent.ACTION_DOWN) {
        // Bắt đầu gửi âm thanh
        sendAudioThread = new SendAudioThread(socket.getOutputStream());
        sendAudioThread.startRecording();
        sendAudioThread.start();
        tvStatus.setText("🎙 Đang nói...");

    } else if (event.getAction() == MotionEvent.ACTION_UP) {
        // Dừng gửi
        sendAudioThread.stopRecording();
        tvStatus.setText("Connected");
    }
    return true;
});
```

✅ Lifecycle & Cleanup — Hoàn thiện

Cập nhật UI từ background thread

```
// Dùng cách này để update TextView từ Thread khác
runOnUiThread(() -> tvStatus.setText("Connected to " + device.deviceName));
```

onDestroy() — Dọn dẹp tài nguyên

```
@Override
protected void onDestroy() {
    super.onDestroy();

    // Dừng audio threads
    if (sendAudioThread != null)    sendAudioThread.stopRecording();
    if (receiveAudioThread != null) receiveAudioThread.interrupt();
```

```java

    // Đóng socket
    try {
        if (socket != null)       socket.close();
        if (serverSocket != null) serverSocket.close();
    } catch (IOException e) { e.printStackTrace(); }

    // Rời nhóm Wi-Fi Direct, giải phóng tài nguyên
    manager.removeGroup(channel, null);
}
```

📅 Timeline gợi ý

Ngày	Việc làm	Milestone kiểm tra
1	Setup project, UI, xin quyền	App hiện UI, xin quyền thành công
2	BroadcastReceiver, quét thiết bị	Thấy được danh sách thiết bị Wi-Fi Direct
3	Kết nối Socket Server/Client	2 máy kết nối được với nhau
4	Audio streaming + Push-to-Talk	Giọng nói truyền được 2 chiều
5	Cleanup, test, fix bug	App hoạt động ổn định, không crash

⚠️ Những lỗi phổ biến cần tránh

1. **Quên** `CHANGE_WIFI_STATE` trong manifest → crash khi gọi `manager.discoverPeers()`
2. **Chạy Socket trên Main Thread** → `NetworkOnMainThreadException`
3. **Không** `unregisterReceiver` trong `onPause()` → memory leak
4. `isGroupOwner` **check sai** → cả 2 máy cùng làm Server hoặc cùng làm Client
5. **Không dùng** `volatile` cho biến `isRecording` → thread không đọc được giá trị mới nhất
6. **Không đóng Socket trong** `onDestroy()` → kết nối treo, lần sau không connect được